

# NLU Course Project - Part 2: NLU

Roberto Savi (257858)

University of Trento

roberto.savi@studenti.unitn.it

## 1. Introduction

This report details the implementation and results for the second part of the NLU course project on intent classification and slot filling. The former consists of classifying the intention of a sentence, while the latter assigns a label to each token to identify a specific part of the sentence. The work consists of two parts:

- **Part A:** Increase the performance of a baseline model for Intent And Slot classification (IAS), by sequentially incorporating bidirectionality and dropout layers.
- **Part B:** Start with a pre-trained BERT model, adapt the code to handle sub-tokenization, and fine-tune the model for the joint task.

Throughout the project, results were evaluated using accuracy for intent classification and F1 score for slot filling.

## 2. Implementation details

Before applying each optimization technique, a 20-trial Optuna sweep [1] using a Tree-structured Parzen Estimator (TPE) was conducted to tune the following hyperparameters: hidden size, embedding size, dropout, and learning rate. Core training settings remained fixed: 200 epochs, patience of 5, gradient clipping of 5, and batch sizes of 64 for training and 128 for evaluation. Each configuration was run 5 times and the results are the average of these five runs. The best configuration was selected based on the highest validation set F1 score, ensuring no data leakage from the test set. Parameters, losses, and loss plots were saved for every configuration. Additionally, the model weights were saved for the best configuration, allowing for result verification via a dedicated evaluation mode.

**Part A** was implemented by:

1. Introducing bidirectionality by setting a flag in the LSTM initialization, which required doubling the hidden size to account for both the forward and backward passes.
2. Implementing two dropout layers: one after the embedding layer and another right before the linear outputs for both slot and intent classification.

**Part B** has been implemented by following the approach proposed by Chen et al. in their paper [2]: exploiting a pre-trained BERT model, specifically `bert-base-uncased`, to jointly perform intent classification and slot filling. The implementation was adapted as follows: Following the paper's methodology, the model predicts the sequence intent by extracting the final hidden state of the `[CLS]` token and passing it through a linear classification layer. Simultaneously, slot filling predictions are generated by feeding the final hidden state of each individual token into a separate linear classification layer.

A core challenge when adapting BERT for sequence labeling is dealing with WordPiece tokenization, which frequently

splits single words into multiple sub-tokens. To maintain compatibility with the original word-level slot labels, the authors recommend using only the hidden state corresponding to the first sub-token of a word for the slot classifier. This process is managed via the `AutoTokenizer` from the Hugging Face Transformers library. By tracking the original word boundaries, the true slot label is assigned exclusively to the initial sub-token of a word. All subsequent sub-tokens, as well as the special `[CLS]` and `[SEP]` tokens, are masked out by assigning them a `[PAD_TOKEN]`. An example is provided in Table 1. Finally, a binary attention mask is generated to ensure that the model completely ignores these padded elements during the forward pass.

## 3. Results

The results demonstrated the efficacy of the optimization techniques in increasing both intent accuracy and F1 score in the LSTM IAS model. The best-performing models across the different parts did not share the same embedding and hidden sizes. Additionally, optimal learning rates remained consistently low across all configurations, as more aggressive rates tended to cause the models to diverge during training. The following sections provide insights and conclusions for each part.

**Part A:** All steps progressively increased intent accuracy and F1 score. Results are shown in Table 2.

- **Bidirectionality:** Processing sentences in both directions improved context awareness. This helped the model classify earlier words based on later ones, raising intent accuracy to  $0.949 (\pm 0.0025)$  and F1 score to  $0.945 (\pm 0.0010)$ .
- **Dropout:** Further introducing dropout layers after the embedding and before the output (both set to a high probability of 0.5) yielded the best results, achieving an intent accuracy of  $0.957 (\pm 0.0033)$  and an F1 score of  $0.951 (\pm 0.0012)$ . This heavy regularization effectively mitigated overfitting.

**Part B:** The fine-tuned, pre-trained BERT model achieved the best overall results across the entire study, with an intent accuracy of  $0.971 (\pm 0.0034)$  and an F1 score of  $0.956 (\pm 0.0020)$ . Results are reported in Table 3.

The significant performance gap between the LSTM and the BERT models is largely due to BERT's deep transformer architecture and its extensive pre-training on massive text datasets. This pre-training provides the model with a strong foundational understanding of language, directly addressing the poor generalization often seen when training NLU models from scratch on small datasets. Furthermore, BERT's self-attention mechanism allows it to effectively capture the structural relationships between the related tasks of overall intent classification and individual slot filling.

## 4. References

- [1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2019, pp. 2623–2631. [Online]. Available: <https://doi.org/10.1145/3292500.3330701>
- [2] Q. Chen, Z. Zhuo, and W. W. Wang, “Bert for joint intent classification and slot filling,” *arXiv preprint arXiv:1902.10909*, 2019.

**Original sentence:**

i want to fly from indianapolis to seattle round trip with the cheapest fare

| Token          | Aligned Label       |
|----------------|---------------------|
| [CLS]          | PAD.TOKEN           |
| [i]            | O                   |
| [want]         | O                   |
| [to]           | O                   |
| [fly]          | O                   |
| [from]         | O                   |
| [indianapolis] | B-fromloc.city_name |
| [to]           | O                   |
| [seattle]      | B-toloc.city_name   |
| [round]        | B-round_trip        |
| [trip]         | I-round_trip        |
| [with]         | O                   |
| [the]          | O                   |
| [cheap]        | B-cost_relative     |
| [##est]        | PAD.TOKEN           |
| [fare]         | O                   |
| [SEP]          | PAD.TOKEN           |

Table 1: Example of slot label alignment for BERT WordPiece tokenization.

| Model              | Hid size | Emb size | Lr     | Emb dropout | Out dropout | Intent Accuracy ( $\pm$ std) | F1 Score ( $\pm$ std) |
|--------------------|----------|----------|--------|-------------|-------------|------------------------------|-----------------------|
| Baseline IAS       | 400      | 350      | 0.0005 | –           | –           | 0.941 (0.0047)               | 0.928 (0.0028)        |
| + Bidirectionality | 300      | 500      | 0.001  | –           | –           | 0.949 (0.0025)               | 0.945 (0.0010)        |
| + Dropout          | 300      | 400      | 0.001  | 0.5         | 0.5         | <b>0.957 (0.0033)</b>        | <b>0.951 (0.0012)</b> |

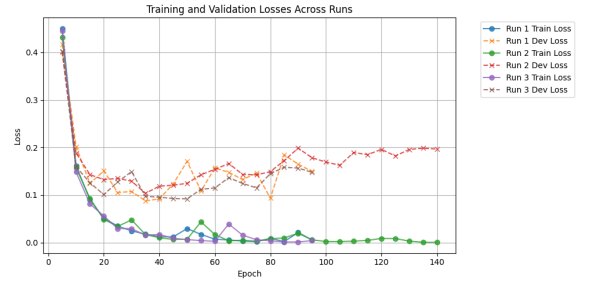
Table 2: Part A: Hyperparameter values, intent accuracy and F1 score on test dataset for the best saved models. The best result is highlighted.

| Model             | Hid size | Emb size | Lr      | Intent dropout | Slot dropout | Intent Accuracy ( $\pm$ std) | F1 Score ( $\pm$ std) |
|-------------------|----------|----------|---------|----------------|--------------|------------------------------|-----------------------|
| bert-base-uncased | 768      | 768      | 0.00005 | 0.1            | 0.2          | <b>0.971 (0.0034)</b>        | <b>0.956 (0.0020)</b> |

Table 3: Part B: Hyperparameter values, intent accuracy and F1 score on test dataset for the best saved models. The best result is highlighted.



(a) Training and validation losses for IAS model with Bidirectionality and Dropout.



(b) Training and validation losses for pre-trained bert-base-uncased.

Figure 1: Loss plots of the best performing models for both parts.